



Universidade do Minho

Projeto Individual

Laboratórios de Estatística II - Licenciatura em Estatística Aplicada
Ano Letivo 2025/2026

Edgar Campos Freitas - A109891

Abril 2026

Jogo dos Sapos

Índice

1.	Introdução	3
2.	Algoritmo - Estratégia de Resolução	3
2.1	Pseudocódigo do Algoritmo	3
2.2	Justificação da Abordagem	3
3.	Vantagens e Limitações	3
3.1	Vantagens	3
3.2	Limitações	4
3.3	O que aconteceria com uma lista de 100 000 blocos?	4
4.	Funcionalidades Extra	4
4.1	Visualização Gráfica com Matplotlib	4
4.2	Menu Interativo	5
5.	Testes	5
5.1	Como foram definidos os testes	5
5.2	Resultados dos Testes	5
6.	Conclusão	6

1. Introdução

Este projeto consiste na resolução de um problema de otimização envolvendo dois sapos que, após uma discussão, pretendem ficar o mais afastados possível um do outro. Os sapos encontram-se inicialmente no mesmo bloco, numa fila de n blocos com diferentes alturas numerados de 0 a n . A regra de movimento é restrita: um sapo só pode saltar para um bloco adjacente se esse bloco tiver uma altura igual ou superior ao bloco onde se encontra. O objetivo é determinar a maior distância possível entre os dois sapos, definida como a diferença entre os seus índices finais.

A solução foi implementada em Python, incluindo visualização gráfica com matplotlib e um menu interativo que permite testar listas manuais, aleatórias e predefinidas.

2. Algoritmo - Estratégia de Resolução

A abordagem escolhida consiste em testar cada bloco como possível ponto de partida. Para cada bloco de início, um sapo expande-se para a esquerda e outro para a direita, avançando enquanto o bloco adjacente tiver altura igual ou superior. A distância entre as posições finais dos dois sapos é calculada e comparada com o máximo encontrado até ao momento.

2.1 Pseudocódigo do Algoritmo

```
for inicio in range(n):
    esquerda = inicio
    while esquerda > 0 and blocos[esquerda-1] >= blocos[esquerda]:
        esquerda = esquerda - 1
    direita = inicio
    while direita < n-1 e blocos[direita+1] >= blocos[direita]:
        direita = direita + 1
    distancia = direita - esquerda
    atualizar max_distancia if distancia > max_distancia
```

2.2 Justificação da Abordagem

Esta estratégia foi escolhida por ser intuitiva e direta: simula exatamente o que os sapos fariam na realidade. Em vez de testar todas as combinações possíveis de pares de blocos (o que seria muito mais complexo), parte-se do princípio de que cada sapo vai o mais longe possível a partir do ponto de partida. Isto reduz significativamente o número de operações necessárias sem perder a garantia de encontrar a solução ótima.

3. Vantagens e Limitações

3.1 Vantagens

- Fácil de compreender e de implementar.

- Não testa combinações desnecessárias: para cada ponto de partida, os sapos expandem-se até ao limite possível.
- Funciona corretamente para qualquer estrutura de blocos: listas crescentes, decrescentes, com zeros, com planaltos ou com grandes variações de altura.
- Robusto: inclui validação de entrada e tratamento de erros.

3.2 Limitações

- O programa testa todos os blocos como ponto de partida e, para cada um, percorre os blocos vizinhos até ao limite. Isto significa que quanto maior for a lista, mais trabalho o programa tem. Ou seja, para listas muito grandes, isto pode tornar a execução mais lenta.
- A solução não aceita alturas negativas, o que é uma decisão justificada: alturas são sempre não negativas.

3.3 O que aconteceria com uma lista de 100 000 blocos?

Com uma lista de 100 000 blocos, o programa teria de testar cada um desses blocos como ponto de partida e, para cada um, percorrer potencialmente muitos outros blocos para a esquerda e para a direita. No pior caso, isto poderia significar milhões ou biliões de operações, tornando a execução muito lenta - potencialmente vários minutos ou até horas.

Uma forma de melhorar isto seria fazer apenas duas passagens pela lista: uma da esquerda para a direita e outra da direita para a esquerda, calculando em cada passagem o maximo alcançável a partir de cada posição. Desta forma, o programa faria muito menos operações e conseguiria responder rapidamente mesmo para listas de grande dimensão.

4 Funcionalidades Extra

Para além da função principal `solucao()`, foram implementadas as seguintes funcionalidades adicionais:

4.1 Visualização Gráfica com Matplotlib

Cada chamada à função `solucao()` gera automaticamente um gráfico de barras que representa visualmente os blocos e a solução encontrada. A legenda do gráfico utiliza um código de cores:

Cor	Significado
Azul	Blocos neutros - não fazem parte da solução
Verde	Posição final dos sapos (melhor esquerda e melhor direita)
Amarelo	Bloco inicial de onde os sapos partem
Marcador verde (●)	Sapo 1 - posição final à esquerda

Marcador vermelho (■)	Sapo 2 - posição final à direita
-----------------------	----------------------------------

Nota: quando o bloco inicial coincide com a posição final de um dos sapos, o bloco fica a amarelo para dar destaque ao ponto de partida e a legenda é atualizada dinamicamente para o indicar.

4.2 Menu Interativo

Foi desenvolvido um menu interativo com três modos de teste:

Opção 1 - Teste manual: o utilizador introduz os valores da lista um a um, escrevendo 'fim' para terminar. O programa valida cada entrada e garante que a lista tem pelo menos 2 blocos antes de executar a função.

Opção 2 - Teste aleatório: o utilizador define o número de blocos e a altura máxima, e o programa gera automaticamente uma lista aleatória usando a biblioteca random.

Opção 3 - Listas predefinidas: submenu com 7 listas de teste prontas a usar, incluindo os dois exemplos do enunciado e listas mais complexas.

5 Testes

Relativamente a este tema, foram realizados 7 cenários de teste, escolhidos para cobrir diferentes comportamentos do algoritmo. Os dois primeiros são os exemplos do enunciado, usados para validar a correção da solução. Os restantes cobrem casos especiais e listas mais complexas.

5.1 Como foram definidos os testes

Para os dois primeiros testes, o resultado esperado é dado pelo enunciado, o que permite confirmar diretamente a correção da função. Para os restantes, o resultado foi verificado manualmente percorrendo o algoritmo passo a passo: para cada bloco de partida, expandiu-se para a esquerda e para a direita seguindo as regras de salto, e calculou-se a distância máxima.

5.2 Resultados dos Testes

#	Lista de Blocos	Resultado Esperado	Cenário Testado
1	[2, 6, 8, 5]	2	Exemplo do enunciado 1
2	[1, 5, 5, 2, 6]	3	Exemplo do enunciado 2
3	[100, 2, 25, 75, 34, 54, 175, 4, 480, 348, 5]	3	Lista com valores muito grandes
4	[3, 0, 123, 75, 34, 96]	3	Zero no meio
5	[24, 63, 3, 643, 263, 346, 12]	2	Grande variação de alturas
6	[0, 1, 0, 50, 51, 52, 1, 2, 3, 0, 100, 200, 150, 0, 1]	4	Zeros como barreiras + subida longa

7	[2, 1, 0, 5, 3, 2, 0, 8, 9, 10, 6, 4, 1, 0, 15, 20, 25, 10, 3, 1]	7	Lista longa e complexa
---	---	---	------------------------

Nota: os tempos de execução são da ordem dos microssegundos (0.000001s a 0.000010s) para todas as listas acima, confirmando a eficiência do algoritmo para listas de pequena dimensão.

6. Conclusão

Sendo assim, a solução desenvolvida resolve corretamente o problema do Jogo dos Sapos para qualquer estrutura de blocos com alturas não negativas. O algoritmo é simples, eficiente para listas de dimensão razoável, e os resultados são apresentados de forma visual e intuitiva através de gráficos gerados automaticamente.

As funcionalidades extra implementadas, visualização gráfica, menu interativo, teste aleatório e listas predefinidas, enriquecem significativamente a usabilidade do programa e tornam-no acessível a qualquer utilizador, independentemente do seu conhecimento técnico.

Por fim, para trabalho futuro, a principal melhoria seria otimizar o algoritmo fazendo apenas duas passagens pela lista em vez de testar todos os pontos de partida, tornando-o capaz de lidar eficientemente com listas de 100 000 blocos ou mais.